Experiment No. 9: Create an ERC-20 Token in Remix IDE

Name: Khushi Singh **Roll No.:** 30

Class: D20A

1. Experiment Title

Experiment No. 9: Create an ERC-20 Token in Remix IDE

2. Aim

To create, deploy and interact with an ERC-20 token using Solidity, Remix IDE, MetaMask and the Sepolia test network.

3. Theory

3.1 Blockchain, Smart Contracts and Ethereum

A blockchain is a distributed, tamper-evident ledger agreed upon by many nodes. A **smart contract** is a program stored on the blockchain that runs automatically when called by a transaction. Ethereum runs contracts on the Ethereum Virtual Machine (EVM), usually written in Solidity, and every transaction costs gas paid in ETH. **Test networks** such as **Sepolia** behave like the main network but use valueless test ETH (obtained from a faucet), so contracts can be tested without risk.

3.2 ERC-20 Token Standard

ERC-20 is the standard interface for **fungible tokens**, in which every unit is identical. Its main elements are:

- **Name, symbol, decimals:** the token's title (Khushi Token), ticker (KHT) and number of decimal places. With 18 decimals, 1 token = 10^{18} base units.
- **Total supply:** total tokens in existence, returned by `totalSupply()`.
- **balanceOf(address):** balance of an address. **transfer(to, amount):** sends tokens from the caller.
- **approve(spender, amount):** lets a spender withdraw up to the amount; **allowance(owner, spender):** returns the amount still permitted.

3.3 Role of Remix IDE and MetaMask

Remix IDE is a browser-based environment to write, compile, deploy and test Solidity contracts. **MetaMask** is a browser-extension wallet that holds the account and asks the user to confirm each transaction; with the *Browser Extension* environment, Remix sends its transactions to MetaMask.

3.4 ERC-20 versus ERC-721

ERC-20 tokens are fungible and recorded as balances. ERC-721 tokens are **non-fungible**: each has a unique ID and owner (e.g. collectibles). ERC-20 tracks *how many* tokens are held; ERC-721 tracks *which* unique tokens are owned.

3.5 Token Extensions

Mintable tokens allow new tokens to be created later, **burnable** tokens allow holders to destroy tokens, and **pausable** tokens allow transfers to be halted in an emergency. They are not used here; this contract mints the whole supply once in its constructor.

4. Tools and Requirements

Tool / Requirement	Use in this experiment
Remix IDE	Write, compile and deploy the contract; call its functions
Solidity compiler (Remix)	Compile MyToken.sol (`pragma solidity ^0.8.20`)
OpenZeppelin Contracts 5.0.2	ERC20 base contract imported in the code
MetaMask extension	Wallet connection and transaction confirmation
Sepolia test network and test ETH	Network used for deployment and gas fees

5. Procedure / Steps

1. **Open Remix IDE and create MyToken.sol.** Open Remix IDE in the browser and create the file `MyToken.sol`.
2. **Write the contract.** Write the ERC-20 contract using OpenZeppelin (Section 6, Figure 1).
3. **Compile.** Compile with the Solidity compiler. No compiler-output screenshot is attached; successful compilation is shown by the successful deployment that follows.
4. **Connect to MetaMask and select Sepolia.** In Deploy & Run, choose the **Browser Extension** environment, connect MetaMask and select the **Sepolia** network (Figure 2).
5. **Deploy.** Enter `initialSupply` = **1000** and click Deploy.
6. **Confirm the transaction.** Review the request in MetaMask (Sepolia, fee 0.0025 SepoliaETH) and click Confirm (Figure 3). The transaction shows **Confirmed** (Figure 4).
7. **Test the read functions.** Call `name()`, `symbol()`, `decimals()`, `totalSupply()` and `balanceOf()` on the deployed contract (Figures 5 to 9).
8. **Transfer tokens.** Call `transfer()` with a recipient and 100 tokens (100 x 10^{18} base units), confirm in MetaMask, then check `balanceOf()` (Figures 10 to 12).
9. **Approve and check allowance.** Call `approve()` for 50 tokens, confirm the Spending cap request, then call `allowance()` (Figures 13 to 15).

6. Program

The complete Solidity source code of `MyToken.sol` used in the experiment (as seen in the Remix editor, Figure 1) is given below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts@5.0.2/token/ERC20/ERC20.sol";

contract MyToken is ERC20 {
    constructor(uint256 initialSupply)
        ERC20("Khushi Token", "KHT")
    {
        _mint(msg.sender, initialSupply * 10 ** decimals());
    }
}
```



Figure 1: MyToken.sol (ERC-20 contract) in the Remix IDE editor.

Explanation of the code

- **Import:** `import "@openzeppelin/contracts@5.0.2/token/ERC20/ERC20.sol";` loads the OpenZeppelin ERC-20 implementation, which provides all the standard functions.
- **Contract declaration:** `contract MyToken is ERC20` inherits that functionality.
- **Constructor:** `constructor(uint256 initialSupply)` runs once at deployment (1000 here).
- **Name and symbol:** `ERC20("Khushi Token", "KHT")` sets the token name and symbol.
- **_mint():** `_mint(msg.sender, initialSupply * 10 ** decimals())` credits the deployer with 1000 x 10^{18} base units (1000 tokens, as decimals() is 18).

7. Output and Observations

7.1 Deployment on Sepolia

Expected: deployment on Sepolia with initial supply 1000. **Observed:** Remix uses the Browser Extension environment with initialSupply = 1000; MetaMask shows the deployment request (fee 0.0025 SepoliaETH) and the transaction is Confirmed.

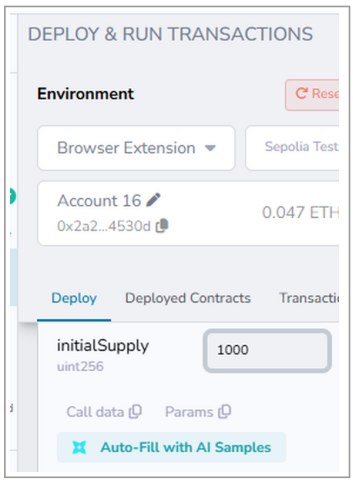


Figure 2: Remix Deploy & Run panel: Browser Extension, Sepolia, initialSupply = 1000.

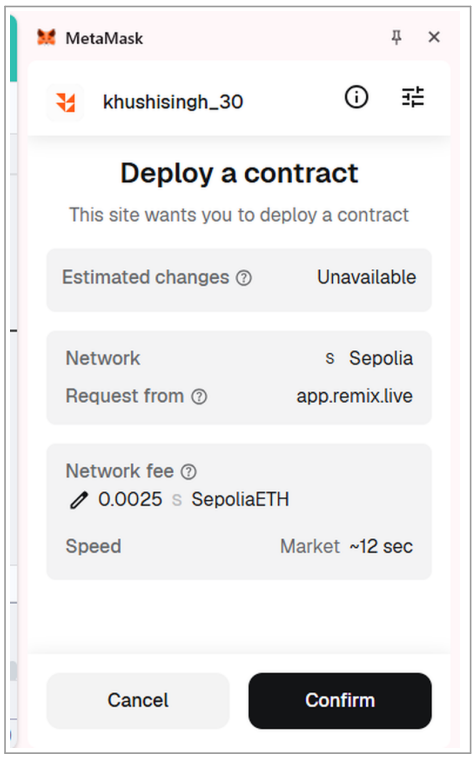


Figure 3: MetaMask "Deploy a contract" confirmation on Sepolia (fee 0.0025 SepoliaETH).

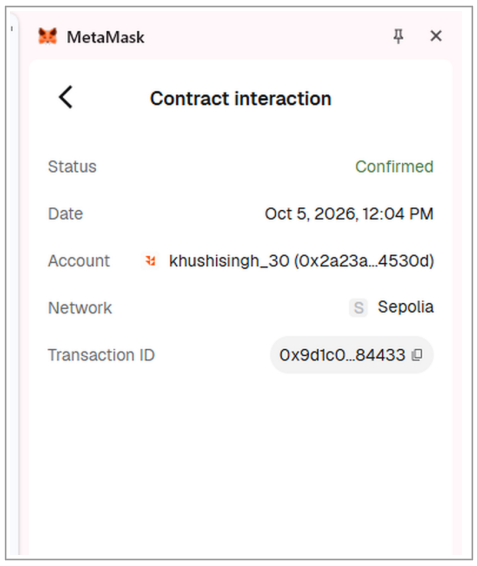


Figure 4: Deployment transaction with status Confirmed.

7.2 Token Details and Balance

Observed: name() = *Khushi Token*, symbol() = *KHT*, decimals() = *18*; totalSupply() and the deployer's balanceOf() both return 1000000000000000000000 base units (1000 tokens).



Figure 5: Output of name().

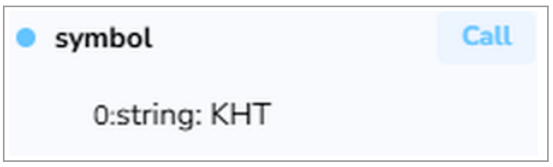


Figure 6: Output of symbol().



Figure 7: Output of decimals().

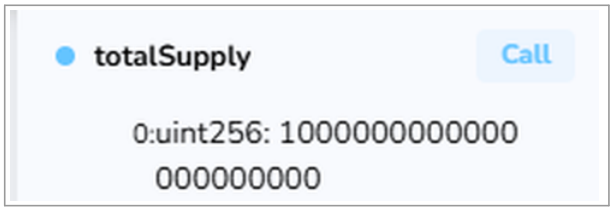


Figure 8: Output of totalSupply().

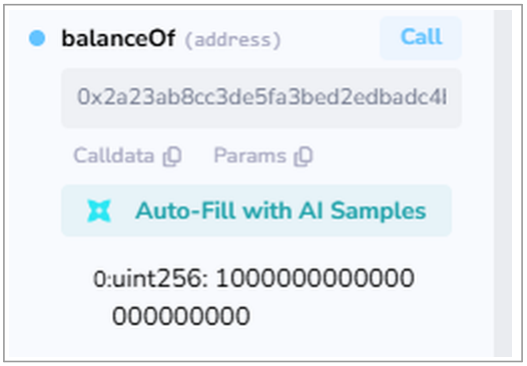


Figure 9: Output of balanceOf() for the deployer (before transfer).

7.3 Token Transfer

Expected: 100 tokens move to the recipient and the deployer balance falls. **Observed:** MetaMask shows "Sending 100" to 0x5Aa93796314f...2b16B446D1; the deployer's balanceOf() afterwards returns 800000000000000000 base units (800 tokens) (see notes).

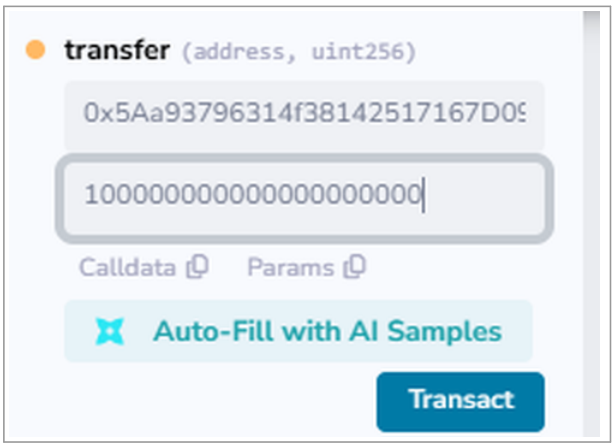


Figure 10: transfer() with recipient address and amount entered.

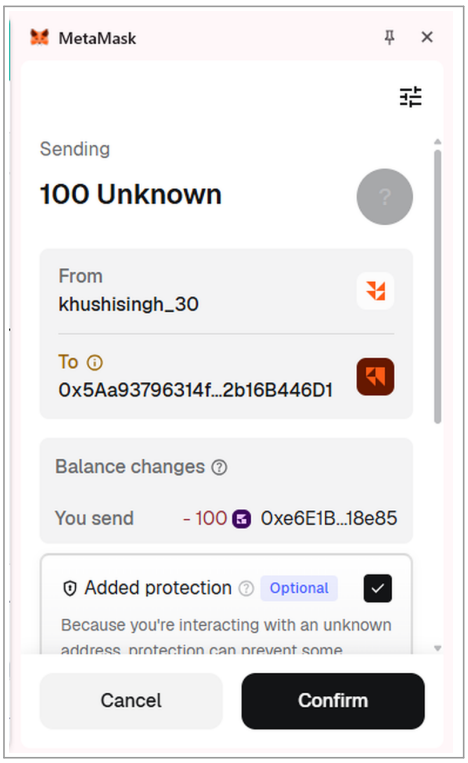


Figure 11: MetaMask confirmation for sending 100 tokens.

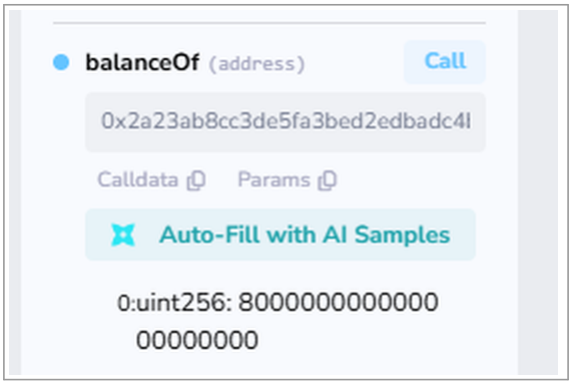


Figure 12: Output of balanceOf() for the deployer after the transfer.

7.4 Approval and Allowance

Observed: approve() was entered for 50 tokens and MetaMask shows a Spending cap request of 50 for the spender; allowance() then returns a non-zero value (see notes).

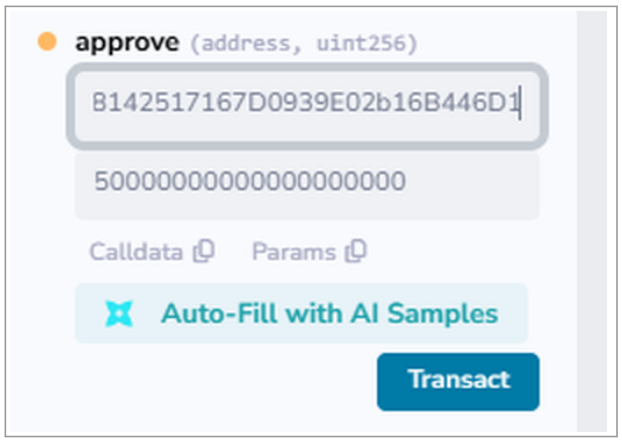


Figure 13: approve() with spender address and amount entered.

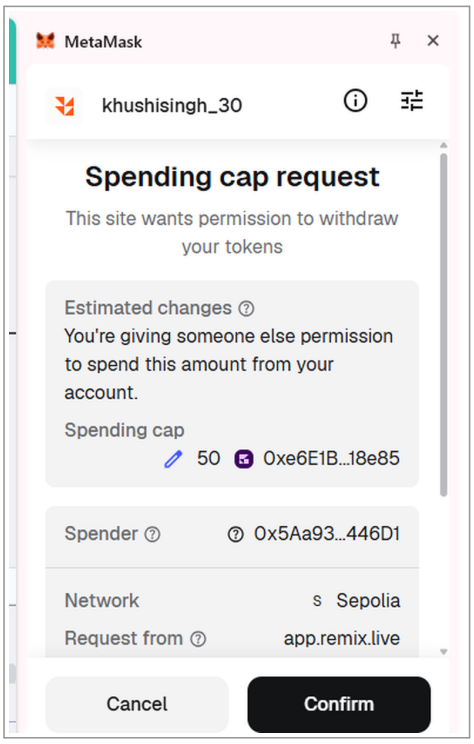


Figure 14: MetaMask Spending cap request (cap of 50).

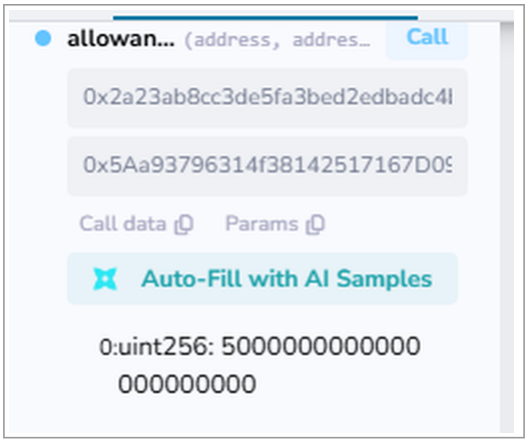


Figure 15: Output of allowance() for the owner and spender.

7.5 Summary of Observations

Parameter	Observed value (from screenshots)	Figure
Environment / network	Browser Extension (MetaMask) / Sepolia	2
Constructor argument	initialSupply = 1000	2
Deployment fee shown	0.0025 SepoliaETH (MetaMask estimate)	3
Deployment transaction	Confirmed; Oct 5, 2026, 12:04 PM	4
Token contract address	Only shortened form visible: 0xe6E1B...18e85	-
name() / symbol() / decimals()	Khushi Token / KHT / 18	5, 6, 7
totalSupply()	10000000000000000000 (1000 tokens)	8
balanceOf(deployer), before transfer	10000000000000000000 (1000 tokens)	9
Transfer	100 tokens sent to 0x5Aa93796314f...2b16B446D1	11
balanceOf(deployer), after transfer	80000000000000000000 (800 tokens)	12
Approval	Spending cap of 50 for spender 0x5Aa93...446D1	13, 14
allowance(owner, spender)	Non-zero value returned; see note	15

Notes on the recorded values

- The full token contract address is not visible in the screenshots (only 0xe6E1B...18e85) and has not been reconstructed; insert it from Remix's Deployed Contracts section.
- Only one 100-token transfer is shown, but the balance afterwards is 800 tokens (200 less than 1000). Add a screenshot of any second transfer or re-check this value.
- The approval is for 50 tokens (Figures 13, 14), but the allowance() output (Figure 15) has 22 digits, i.e. 5000 tokens. Re-capture allowance() if required.
- Verification of the contract on Etherscan was not performed in the attached evidence.

8. Conclusion

An ERC-20 token, **Khushi Token (KHT)**, was created in Solidity with OpenZeppelin and deployed on the Sepolia test network using Remix IDE and MetaMask, with an initial supply of 1000 tokens. The functions name(), symbol(), decimals(), totalSupply() and balanceOf() returned the expected values, and transfer(), approve() and allowance() were used to interact with the token.